

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EduCore - Async & Fetch</title>
  <link rel="stylesheet" href="styles.css">
  <!-- Task 3 Step 55: Axios via CDN -->
  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
</head>
<body>

  <header>
    <h1>EduCore Student Portal</h1>
  </header>

  <main>

    <section id="courses-section">
      <h2>Courses</h2>
      <p id="courses-loading">Loading courses...</p>
      <div class="course-grid" id="course-grid"></div>
      <p id="total-credits"></p>
    </section>

    <section id="notifications">
      <h2>Notifications</h2>
      <p id="notif-loading">Loading...</p>
      <div id="notif-error"></div>
      <div id="notif-list"></div>
    </section>

    <section id="axios-section">
      <h2>Axios — User 1 Posts</h2>
      <div id="axios-posts"></div>
    </section>

  </main>

  <script type="module" src="app.js"></script>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body { font-family: Arial, sans-serif; font-size: 16px; background: #f5f5f5; color: #333; }

header { background: #1e3a5f; color: #fff; padding: 16px 24px; }

main { max-width: 960px; margin: 24px auto; padding: 0 16px; }

section { background: #fff; border: 1px solid #ddd; border-radius: 4px; padding: 20px;
margin-bottom: 24px; }

h2 { font-size: 20px; margin-bottom: 12px; color: #1e3a5f; }

.course-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap:
12px; margin-top: 12px; }

.course-card { border: 1px solid #ddd; border-radius: 4px; padding: 12px; background: #fafafa; }

.notif-card { border-left: 3px solid #1e3a5f; padding: 10px 12px; margin-bottom: 10px;
background: #f9f9f9; }

.error-msg { color: #c00; margin-bottom: 8px; }

button { padding: 8px 16px; cursor: pointer; }
```

data.js

```
export const courses = [
  { id: 1, code: 'CS101', name: 'Intro to Programming', credits: 4, grade: 'A' },
  { id: 2, code: 'CS201', name: 'Data Structures', credits: 4, grade: 'B' },
  { id: 3, code: 'MA101', name: 'Calculus I', credits: 3, grade: 'A' },
  { id: 4, code: 'EC101', name: 'Circuit Theory', credits: 3, grade: 'B' },
  { id: 5, code: 'ME101', name: 'Engineering Mechanics', credits: 3, grade: 'C' },
];
```

app.js

```
import { courses } from './data.js';

const BASE = 'https://jsonplaceholder.typicode.com';
```

```
// =====  
// TASK 1 — Promises and async/await  
// =====
```

```
// Step 45: Promise chain
```

```
function fetchUserPromise(id) {  
  return fetch(`${BASE}/users/${id}`)  
    .then(res => res.json())  
    .then(user => console.log('Promise chain - user name:', user.name));  
}  
fetchUserPromise(1);
```

```
// Step 46: same function rewritten with async/await + try/catch
```

```
async function fetchUser(id) {  
  try {  
    const res = await fetch(`${BASE}/users/${id}`);  
    const user = await res.json();  
    console.log('async/await - user name:', user.name);  
  } catch (err) {  
    console.error('fetchUser error:', err.message);  
  }  
}  
fetchUser(1);
```

```
// Step 47: simulated 1-second delay then returns local courses
```

```
function fetchAllCourses() {  
  return new Promise(resolve => setTimeout(() => resolve(courses), 1000));  
}
```

```
// Step 48: show loading, render cards after promise resolves
```

```
const courseGrid = document.getElementById('course-grid');  
const loadingEl = document.getElementById('courses-loading');  
const totalEl = document.getElementById('total-credits');
```

```
fetchAllCourses().then(list => {  
  loadingEl.style.display = 'none';  
  list.forEach(course => {  
    const card = document.createElement('article');  
    card.className = 'course-card';  
    card.innerHTML = `

### ${course.code}</h3><p>${course.name}</p><p>${course.credits}<br>Credits</p>`; courseGrid.appendChild(card); }); const total = list.reduce((sum, c) => sum + c.credits, 0);


```

```
totalEl.textContent = `Total Credits: ${total}`;
});
```

```
// Step 49: Promise.all — fetch two users simultaneously
```

```
Promise.all([
  fetch(`${BASE}/users/1`).then(r => r.json()),
  fetch(`${BASE}/users/2`).then(r => r.json())
]).then(([user1, user2]) => {
  console.log('Promise.all - users:', user1.name, '&', user2.name);
});
```

```
// =====
// TASK 2 — Fetch API with Error Handling
// =====
```

```
// Step 50: reusable fetch wrapper
```

```
async function apiFetch(url) {
  const res = await fetch(url);
  if (!res.ok) {
    throw new Error(`HTTP error ${res.status}: ${url}`);
  }
  return res.json();
}
```

```
const notifLoading = document.getElementById('notif-loading');
const notifError = document.getElementById('notif-error');
const notifList = document.getElementById('notif-list');
```

```
// Step 51 + 52: load posts, show loading spinner, render cards
```

```
async function loadNotifications(url) {
  notifLoading.style.display = 'block'; // show loading
  notifError.innerHTML = "";
  notifList.innerHTML = "";

  try {
    const posts = await apiFetch(url);
    notifLoading.style.display = 'none';
    posts.slice(0, 5).forEach(post => {
      const card = document.createElement('div');
      card.className = 'notif-card';
      card.innerHTML = `${post.title}</strong><p>${post.body.slice(0, 60)}...</p>`;
      notifList.appendChild(card);
    });
  }
```

```

    } catch (err) {
      // Step 53: user-friendly error message in UI
      notifLoading.style.display = 'none';
      notifError.innerHTML = `
        <p class="error-msg">Failed to load: ${err.message}</p>
        <button id="retry-btn" type="button">Retry</button>
      `;
      // Step 54: retry button re-calls the function
      document.getElementById('retry-btn').addEventListener('click', () => {
        loadNotifications(`${BASE}/posts`);
      });
    }
  }

// Load successfully first
loadNotifications(`${BASE}/posts`);

// Uncomment to test Step 53 error + retry:
// loadNotifications(`${BASE}/nonexistent`);

// =====
// TASK 3 — Axios
// =====

// Step 58: request interceptor — logs before every request
axios.interceptors.request.use(config => {
  console.log('API call started:', config.url);
  return config;
});

// Step 56: apiFetch rewritten with axios (auto JSON parse, throws on non-2xx)
async function axiosFetch(url) {
  const res = await axios.get(url);
  return res.data;
}

// Step 57: fetch posts for userId=1 using params object
async function loadAxiosPosts() {
  try {
    const res = await axios.get(`${BASE}/posts`, { params: { userId: 1 } });
    const posts = res.data;
    const container = document.getElementById('axios-posts');
    posts.slice(0, 3).forEach(post => {

```

```

        const card = document.createElement('div');
        card.className = 'notif-card';
        card.innerHTML = `<strong>${post.title}</strong>`;
        container.appendChild(card);
    });
} catch (err) {
    console.error('Axios error:', err.message);
}
}
loadAxiosPosts();

/*
 * Step 59 — fetch vs axios differences:
 *
 * 1. JSON parsing: fetch requires res.json() manually; axios parses automatically (res.data).
 * 2. Error handling: fetch only rejects on network failure, NOT on 404/500 — you must check
 *    response.ok manually. Axios throws automatically on any non-2xx status.
 * 3. Features: Axios has built-in request/response interceptors, timeout support, and
 *    request cancellation. fetch has none of these out of the box.
 */

```

OUTPUT:

EduCore Student Portal

Courses

CS101

Intro to Programming

4 Credits

CS201

Data Structures

4 Credits

MA101

Calculus I

3 Credits

EC101

Circuit Theory

3 Credits

ME101

Engineering Mechanics

3 Credits

Total Credits: 17

ME101

Engineering Mechanics

3 Credits

Total Credits: 17

Notifications

sunt aut facere repellat provident occaecati excepturi optio reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita e...

qui est esse

est rerum tempore vitae sequi sint nihil reprehenderit dolor...

ea molestias quasi exercitationem repellat qui ipsa sit aut

et iusto sed quo iure voluptatem occaecati omnis eligendi au...

eum et est occaecati

ullam et saepe reiciendis voluptatem adipisci sit amet autem...

nesciunt quas odio

repudiandae veniam quaerat sunt sed alias aut fugiat sit aut...

Axios — User 1 Posts

sunt aut facere repellat provident occaecati excepturi optio reprehenderit

qui est esse

ea molestias quasi exercitationem repellat qui ipsa sit aut